

TraceDiff: Anomaly Detection on Microservice Traces Through Frequency-Aware Conditional Diffusion Model

Kaiqi Ding¹, Dezhi Ran², Yuanmu Ma, Zijian Song, Kaigui Bian¹, *Senior Member, IEEE*,
and Tao Xie¹, *Fellow, IEEE*

Abstract—Distributed traces are vital for fine-grained fault diagnosis in microservice systems. However, accurately capturing temporal dependencies and inter-span correlations remains challenging due to the dynamic patterns and temporal sparsity inherent in trace latencies. To address the challenge, we propose *TraceDiff*, a frequency-aware conditional diffusion model that integrates temporal and frequency features for anomaly detection. Specifically, *TraceDiff* first constructs a Trajectory Feature Matrix (TFM) to systematically fuse fine-grained structural and latency features. It then employs a dual-encoder mechanism that combines Global and Local Frequency Encoders alongside an attention mechanism to extract multi-scale frequency features, robustly capturing dynamic patterns. Finally, a conditional denoising diffusion model performs multi-step autoregressive forecasting, effectively mitigating trace sparsity and modeling complex span dependencies. Evaluated on two public datasets and a real-world production system, *TraceDiff* significantly outperforms state-of-the-art anomaly detectors. Compared to the best-performing baselines, it consistently improves F1-scores by 3.22% to 3.42% for component-level and 0.52% to 4.12% for trace-level anomalies, while notably boosting Top-1 root cause identification accuracy by 10.84% to 16.00%.

Index Terms—Diffusion model, trace anomaly detection, root cause analysis, microservice system.

I. INTRODUCTION

MICROSERVICE systems have gained popularity due to their ability to enhance scalability, flexibility, and agile iteration capabilities [1], [2], [3]. Microservice systems decompose software applications into small, lightweight services,

enabling independent modules for concurrent development and rapid updates [2]. However, the fine-grained service decomposition also introduces increased vulnerability to performance anomalies [4]. To maintain system reliability, effective anomaly detection and precise root cause analysis become essential [5].

To enable fine-grained anomaly diagnosis in microservice systems, traces have been widely adopted in industrial detection systems [6], [7], [8]. Traces, constructed from spans that represent distinct operational units, capture the execution latency of external requests and microservice API interactions [9]. By analyzing span-level timing data, trace latency provides insights into service interaction efficiency and the overall performance of the invocation chain. Operation engineers typically analyze the latency and inter-span correlations [10], [11] within traces to diagnose anomalies.

Beyond performance analysis, trace-based anomaly detection also serves as a vital defensive measure against attacks targeting system availability. Many security threats initially manifest as performance anomalies before escalating to systemic failures. To formalize this observation, we introduce a “symptom-equivalence” framework: sophisticated security attacks (e.g., Application-Layer DDoS, adaptive resource exhaustion, or resource hijacking) inevitably manifest at the trace level as severe structural deviations in Process Time (PT) and Wait Time (WT), symptoms identical to operational faults. Therefore, while *TraceDiff* is fundamentally designed as a service availability and performance evaluator for operational issues, it natively extends to security domains. By accurately detecting these fine-grained latency deviations, *TraceDiff* not only diagnoses operational bottlenecks but also acts as a generalized guard safeguarding system availability against malicious interference.

Drawing from our analysis in a production microservice system and an in-depth review of related literature, we point out two inherent challenges in trace anomaly detection.

- *Challenge 1. Dynamic patterns of latency:* Traces exhibit dynamic pattern shifts caused by factors such as network conditions, system updates, or changes in user behavior. These shifts manifest as trends, periodicity, and fluctuations over time, resulting in irregular time series. Existing approaches of anomaly detection [7], [9], [12], [13] often struggle with irregular data due to their limited sensitivity in identifying such inconsistencies, frequently

Received 14 January 2026; revised 3 July 2026; accepted 6 July 2026. Date of publication 13 July 2026; date of current version 1 September 2026. This work was supported in part by The Fundamental Research Funds for the Central Universities, Peking University, in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant JYB2025XDXM118, and in part by the National Natural Science Foundation of China under Grant U24A20235, Grant 623B2006, Grant 92464301, and Grant U25A6023. Recommended for acceptance by I. Karim. (Corresponding author: Kaigui Bian.)

Kaiqi Ding, Zijian Song, and Kaigui Bian are with the School of CS, Peking University, Beijing 100871, China, and also with the Beijing Key Laboratory of Software and Hardware Cooperative Artificial Intelligence Systems, Beijing 100871, China (e-mail: bkg@pku.edu.cn).

Dezhi Ran and Tao Xie are with the Key Laboratory of High Confidence Software Technology, Ministry of Education, Peking University, Beijing 100871, China.

Yuanmu Ma is with the Beijing Institute of Technology, Beijing 100811, China.

Digital Object Identifier 10.1109/TDSC.2026.3712748

misclassifying them as noise. The misclassification can obscure genuine anomalies, thereby reducing detection accuracy. Effective anomaly detection approaches must be adaptable to changes in data distribution and capable of accurately capturing dynamic variations in temporal patterns.

- *Challenge 2. Temporal sparsity of latency:* Traces often exhibit significant sparsity along the temporal dimension. For instance, certain microservices may only be invoked during specific business processes, resulting in infrequent invocations or even no records during other processes. This sparsity introduces discontinuities in traces, posing challenges for deep learning approaches [8], [9], [14], which typically rely on the assumption of uniformly distributed data. Moreover, when traces are extremely sparse, the relative impact of noise becomes more pronounced. In the latent space, the noise can be further amplified, significantly impairing the model's learning effectiveness and performance.

Existing approaches [15], [16] employ adversarial training frameworks to filter out noisy data, while WinG [17] adopts a warping procedure to assess an element's anomaly severity based on its status deviation, thereby mitigating the interference caused by variable invocations. However, these approaches typically focus solely on time-domain information. In traces, outliers or normal fluctuation can significantly disrupt model training. In contrast, frequency-domain information can effectively mitigate such disruptions through filtering and adaptive frequency component processing [18], [19], thereby enhancing the model's robustness and stability (**Challenge 1**). While some approaches [8], [9], [15] focus on employing Variational Autoencoders (VAEs) [20] to reconstruct dependencies from temporal traces, with anomalies identified based on the discrepancies between observed and reconstructed behaviors. However, VAEs typically assume that the data is uniformly distributed and that the latent space follows a Gaussian distribution, which may not effectively capture the complex distribution characteristics of sparsity latency (**Challenge 2**). Therefore, we utilize diffusion models [21], [22], adept at modeling complex dependencies in temporal sequences, to overcome this challenge. Diffusion models can integrate conditional information during the generation process [23], such as textual descriptions or other prior knowledge, enabling them to capture the complex distribution of traces.

In this paper, we propose *TraceDiff*, a novel framework for trace anomaly detection based on frequency-aware conditional denoising diffusion models, systematically addressing the preceding two challenges. TraceDiff performs fine-grained decomposition of the trace structure. It extracts spans based on the invocation sequence, capturing Process Time and Wait Time in the time domain. To address **Challenge 1**, we propose a frequency-aware feature enhancement approach that utilizes a Global Frequency Encoder (GFE) and a Local Frequency Encoder (LFE) for capturing multi-scale frequency features. These dual encoders enable the simultaneous extraction of long-term trends and short-term fluctuations, adapting to the dynamic patterns of latency. Furthermore, an attention mechanism

is integrated to select the most informative features from the frequency domain, ensuring the accurate construction of short-term trends. This mechanism enhances the model's focus on the most relevant information for current predictions, thereby improving its adaptability to dynamic patterns. For **Challenge 2**, TraceDiff employs a diffusion model to perform stochastic modeling of the time series data through a noise/denoising process. This step is designed to capture complex temporal dependencies and inter-span correlations. To further mitigate data sparsity, TraceDiff incorporates relative time embedding and combines time-domain and frequency-domain features as conditions for the diffusion model. This approach enhances the model's ability to effectively handle temporal dynamics and sparse patterns.

We evaluate TraceDiff on two widely used open-source platforms, TrainTicket [24] and SocialNetwork [25], as well as a real-world production microservice system. Experimental results demonstrate that TraceDiff outperforms state-of-the-art anomaly detectors. Compared to the best-performing baselines, the F1 score improves by 3.22% to 3.42% and 0.52% to 4.12% for component-level and trace-level latency anomaly detection, respectively. Additionally, TraceDiff excels in root cause localization, achieving a 10.84% to 16.00% improvement in Top-1 accuracy. In summary, this paper makes the following main contributions:

- *Frequency-aware Architecture:* TraceDiff is the first framework integrating frequency-domain analysis with diffusion models. It systematically captures highly dynamic structural and latency patterns.
- *Sparsity-tolerant Generative Modeling:* We formulate trace anomaly detection as a conditional generative task. The diffusion model interpolates temporal gaps, effectively overcoming trace sparsity and robustly modeling complex inter-span correlations.
- *Real-world Practicality:* TraceDiff delivers critical production value, providing precise Top-1 localization for rapid fault diagnosis, and acting as a frontline guard against availability-targeting security threats.

II. BACKGROUND

A. Microservice Traces

1) *Microservice and API:* In microservice architectures, inter-service communication is primarily facilitated through API invocations between distributed components. Each microservice contains multiple APIs (components), and every API is uniquely identified by a numeric ID, referred to as an API ID.

2) *Trace and Span:* Each trace consists of multiple spans sharing the same trace ID, capturing the interaction between the APIs of microservices. The relationships between spans can be identified using the parent ID, which helps locate the caller-callee hierarchy in Fig. 1.

3) *Process Time and Wait Time:* Trace latency primarily consists of two key components: Process Time (PT) and Wait Time (WT) [15]. Process Time (PT) measures the duration an API takes to handle a request, computed as the difference between its execution end time and start time. It reflects the total time

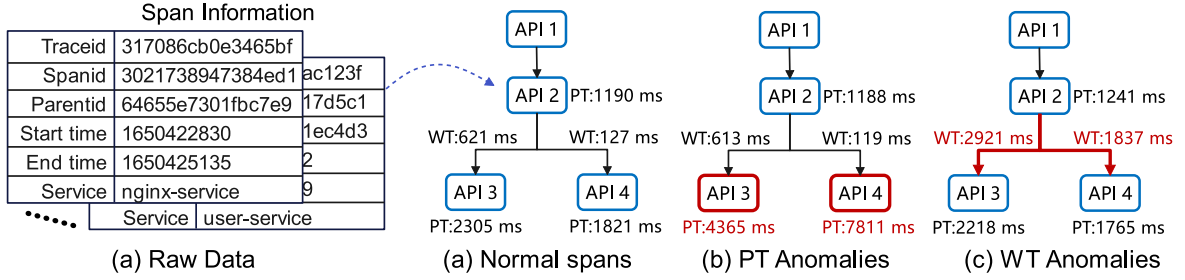


Fig. 1. Example of a trace, its spans, and anomaly types. (a) Span information showing API2 calling API3, with a normal process time (PT) of 2305 ms at API3 and a wait time (WT) of 621 ms for the span API2→API3. (b) PT anomalies on API3 (4365 ms, abnormal compared to 2305 ms) and API4, marked by red circles. (c) WT anomalies on API2→API3 (2921 ms, abnormal compared to 621 ms) and API2→API4, indicated by red arrow lines.

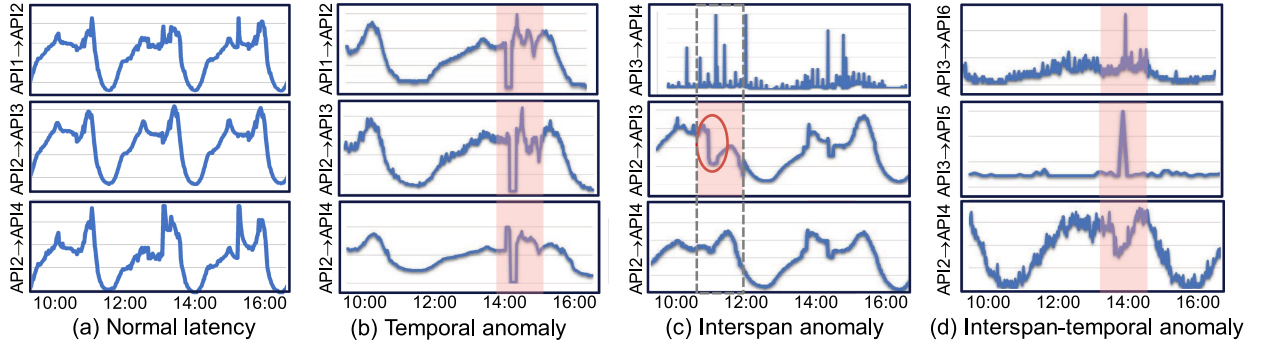


Fig. 2. Illustration of normal latency and latency anomalies types (anomalous segments highlighted in pink shadow). In **Temporal Anomalies**, latency deviates from corresponding time patterns. In **Interspan Anomalies**, the relationships between spans violate the expected correlations (e.g., spans should show a positive correlation but do not exhibit co-growth). In **Interspan-Temporal Anomalies**, the above two types of anomalies occur.

consumed by the API from receiving the request until sending the response, excluding any external delays. Wait Time (WT) quantifies the transmission delay between interdependent microservices, accounting for network latency and queue waiting. It is derived as the interval between the current API's execution end time and the subsequent API's execution start time. This metric encompasses three key components: the current API's response serialization delay, network transmission overhead, and queue wait time before the next API begins processing. Thus, WT represents the cumulative transition latency when a request propagates through the service chain.

B. Traces Anomalies

1) **PT and WT Anomalies:** As previously defined, trace latency anomalies can be classified into PT anomalies and WT anomalies [9], [15].

PT anomalies represent abnormal delays in the internal execution of a service. While often caused by operational faults such as inefficient algorithms or software bugs, they are also critical indicators of resource exhaustion attacks compromising system integrity. For instance, cryptojacking malware competing for CPU resources, or Algorithmic Complexity Attacks (e.g., ReDoS), can force legitimate requests to undergo prolonged execution and frequent context switching. Fig. 1(b) illustrates an example where the PT of API3 and API4 in the anomalous trace escalates rapidly, exceeding the normal range, a pattern

characteristic of both severe resource contention and unauthorized computational consumption.

WT anomalies reflect delays in request transmission and queueing, serving as a primary symptom for availability threats. Beyond standard network issues like packet loss, sharp spikes in WT are frequently triggered by Application-Layer DDoS attacks (e.g., HTTP floods). In such scenarios, a surge of malicious traffic saturates the service request queue, causing legitimate requests to wait excessively before execution. Fig. 1(c) demonstrates a case where the WT of API2→API3 and API2→API4 spikes significantly, surpassing the normal threshold, signaling potential network congestion or a targeted availability attack.

2) **Anomaly Types:** PT and WT anomalies can be further categorized into three types: temporal anomalies, interspan anomalies, and interspan-temporal anomalies, as shown in Fig. 2. **Temporal anomalies** occur when the PT of several spans deviates significantly from historical patterns, often indicating large-scale failures or reboots. In Fig. 2(b), the PT of three spans drops to near-zero (pink shadow), diverging from their normal temporal patterns. **Interspan anomalies** arise when the correlation pattern between spans is violated. For example, in Fig. 2(c), a user surge causes increased processing delays, the WT of AP3→API4, API2→API3 and API2→API4 should increase following a positive correlation, but API2→API3 is blocked and throws an exception, disrupting the expected positive correlation in WT growth in the period 11:00–12:00 (pink shadow). **Interspan-temporal anomalies** involve violations of

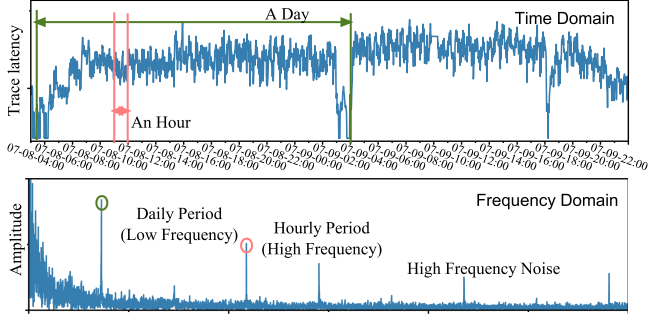


Fig. 3. Trace latency decomposition in time and frequency domains. (Top) Time-domain representation showing trace latency over 48 hours with superimposed daily and hourly patterns. (Bottom) Frequency-domain representation clearly separating the daily periodic component (low frequency) from hourly variations (high frequency). The frequency domain enables isolation of different operational phenomena that are entangled in the time domain.

both temporal and interspan dependencies. In Fig. 2(d), Temporal WT anomalies occur in API3 → API6 and API3 → API5 during 13:00–14:00, while an interspan anomaly affects API2 → API4 (pink shadow).

3) *Root Cause Location*: The root cause is the primary component responsible for trace failures, which can lead to performance degradation, functional issues, and other problems. Due to the complex dependencies between spans, a failure in one API can trigger cascading failures across the system. Therefore, quickly and accurately identifying the root cause is critical for resolving issues effectively and optimizing system performance.

C. Trace Latency in Frequency Domain

Traditional trace analysis primarily operates in the time domain, examining latency values directly as they evolve over time. However, microservice trace latency often exhibits complex temporal patterns comprising multiple overlapping periodicities and trends. The frequency domain provides an alternative representation that offers unique advantages for analyzing such complex signals. As illustrated in Fig. 3, the complex time-domain signal can be decomposed into distinct frequency components, each corresponding to different temporal scales of system behavior.

In microservice systems, specific frequency bands correspond to distinct operational phenomena:

- *Low-frequency components* (e.g., periods of 12–24 hours) typically correspond to daily workload cycles, reflecting predictable patterns of user activity and resource utilization.
- *High-frequency components* (e.g., periods of minutes to seconds) capture transient events such as network jitter, queueing dynamics, or Scheduled tasks (periodic cron jobs or batch processing operations).

The Discrete Fourier Transform (DFT) converts a time-domain signal $x[n]$ of length L into its frequency-domain representation:

$$X[k] = \sum_{n=0}^{L-1} x[n] e^{-2\pi i k n / L}, \quad k = 0, 1, \dots, L-1 \quad (1)$$

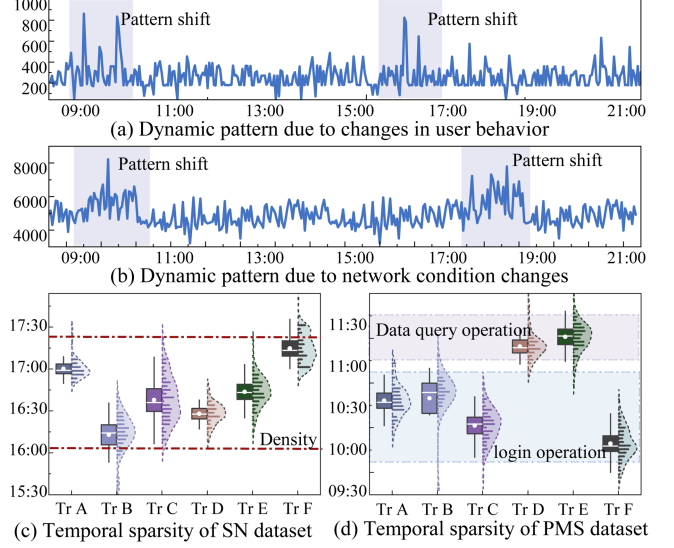


Fig. 4. Analyzing trace data: Confronting the dual challenges of dynamic patterns and temporal sparsity.

where $X[k]$ represents the complex amplitude at frequency index k , and i is the imaginary unit.

The Fast Fourier Transform (FFT) is an efficient algorithm for computing the DFT with time complexity $O(L \log L)$ instead of $O(L^2)$. For real-valued trace latency data, we utilize the Real FFT (rFFT) algorithm, which exploits Hermitian symmetry to compute only the non-redundant positive frequencies, reducing both computational cost and memory usage.

III. MOTIVATION

A. Challenge 1: Dynamic Patterns of Latency

The dynamic nature of microservices architecture inherently results in dynamic patterns of trace latency. For example, sudden changes in user behavior, such as a surge in search requests or transaction operations, can increase service load and significantly impact latency. In Fig. 4(a), shifts in user behavior cause latency spikes and pattern changes. Additionally, network-related factors like bandwidth fluctuations, or unstable connections can further destabilize latency, leading to irregular pattern shifts, as depicted in Fig. 4(b). These variations in latency patterns underscore the inherent dynamism of microservices' operational behavior, posing significant challenges to achieving stable latency detections.

Existing approaches [7], [8], [11], [26], [27] primarily rely on time-domain analysis, which fails to capture dynamic changes in data distribution and may misinterpret irregularities as noise. VAE-based methods typically enforce a fixed standard Gaussian prior on the latent space. This assumption forces the model to “over-smooth” the complex, multi-modal distributions inherent in dynamic microservice traces, effectively treating legitimate sharp pattern shifts as noise or outliers. Consequently, they fail to capture the intricate probability density of shifting latencies. In contrast, TraceDiff utilizes a Denoising Diffusion Probabilistic Model (DDPM) which does not rely on a restricted prior form.

By learning the gradients of the data density function (score matching) through an iterative denoising process, TraceDiff can explicitly model the complex, non-Gaussian distributions of dynamic patterns, ensuring that sudden but legitimate latency shifts are accurately represented rather than discarded.

B. Challenge 2: Temporal Sparsity of Latency

In microservices systems, traces often exhibit significant temporal variability, resulting in sparse and irregularly distributed. For example, certain traces are only frequently invoked during specific business scenarios, leaving minimal or no invocation records during other periods. In Fig. 4(a), the temporal distribution of traces exhibits significant unevenness. During peak business hours, such as between 16:00 and 17:30, traces are dense, while off-peak periods show sparse distribution. Fig. 4(b) further highlights temporal distribution differences, which arise from the varying purposes of traces. For example, traces A, B, C and F, related to login operations, exhibit dense data from 10:00 to 11:00, while traces D and E show dense data from 11:00 to 11:30 for data query operations.

Current approaches [8], [9], [15] often struggle with such sparsity. Deterministic models (like RNNs or Transformers) typically require continuous history for autoregressive forecasting, which is disrupted by the discontinuities in sparse traces. Similarly, VAEs tend to reconstruct sparse inputs poorly because the sparsity leads to a collapse in the latent representation (posterior collapse). TraceDiff addresses this by reformulating trace modeling as a conditional generative task. Unlike deterministic regression, the diffusion process starts from pure noise and iteratively refines the signal conditioned on available structural embeddings. This stochastic generation capability allows the model to robustly “bridge” temporal gaps and infer coherent dependencies even from sparse observations, effectively mitigating the impact of missing data points that handicap traditional probabilistic models.

C. Key Designs of TraceDiff

To address these challenges, we propose TraceDiff, which introduces two key designs:

- *Design 1. Frequency-aware feature enhancement:* We introduce a Global Frequency Information Extraction Module (GFE) and a Local Frequency Information Extraction Module (LFE) to capture both global and local frequency characteristics of the data. This dual approach enables the model to simultaneously capture long-term trends and short-term fluctuations, adapting better to the dynamic nature of the data. Additionally, an attention mechanism is integrated to select the most informative features from the frequency domain, ensuring accurate construction of short-term trends. This mechanism enhances the model’s focus on the most relevant information for current predictions, improving its adaptability to dynamic changes.
- *Design 2. Trace latency modeling with conditional diffusion model:* TraceDiff employs a noise/denoising process to perform stochastic modeling of time series. During the generation process, the model can incorporate temporal

conditions to handle the irregularity of time intervals, thereby better addressing the sparsity of time series data.

IV. PROPOSED APPROACH

Fig. 5 presents the overall architecture of TraceDiff, which consists of *trace preprocessing*, *conditional diffusion model* and *anomaly evaluation and alerting*. Trace preprocessing extracts structural characteristics and latency-related properties from trace data. The conditional diffusion model integrates temporal and frequency information by encoding them as high-dimensional embeddings. These embeddings serve as conditions for a denoising diffusion model, which predicts near-future span response latency in an autoregressive manner. Anomaly evaluation and alerting identify and compute anomalous features, generating actionable alerts for detected anomalies.

A. Trace Preprocessing

The microservice system generates a significant volume of traces, automatically collected and stored in a large database by the tracing system. However, analyzing all the trace data daily is impractical. Therefore, before pre-processing, we perform a fair and random sampling of the trace data, selecting 1% of the total while ensuring that each trace captures the complete sequence of component invocations. This approach provides an unbiased representation of the system’s state [7], while also keeping the pre-processing overhead manageable.

During the preprocessing of the trace data, TraceDiff initially identifies spans within the given trace based on the order of calls, using the TraceID and ParentID. We extract two types of latency features: process time (PT) and wait time (WT). We analyze the sequential characteristics of each invocation pair based on the order of the trace. For instance, the trace in Fig. 1(a) can produce three span records, namely API1 → API2, API2 → API3, API2 → API4, shown as trace structure in Fig. 5(a). We mainly take two spans for example analysis, we capture the PT and WT of the span by consolidating time series data. To facilitate this, we adopt the Trajectory Feature Matrix (TFM) [15], to store the latency features extracted from the trace. TFM contains the fusion of fine-grained structural and latency feature information as we extract the relationship between spans in a trace to become the first dimension (**interspan dependency**), and the latency of each span fluctuates with time in the second dimension (**temporal dependency**).

B. Conditional Diffusion Model

1) *Diffusion Probabilistic Model:* Diffusion models generate samples by gradually reversing a noising process, moving from white noise to the desired signal. These models are trained to reduce noise at each step of this process. When considering this noise removal as a learned diagonal Gaussian, the training loss can be treated as a regression problem.

Let $\mathbf{x}^0 \sim q_{\chi}(\mathbf{x}^0) \in \mathbb{R}^{m \times l}$ denote a TFM containing l spans, where each span is represented by an m -dimensional feature vector. Gaussian noise is gradually added to the input and the

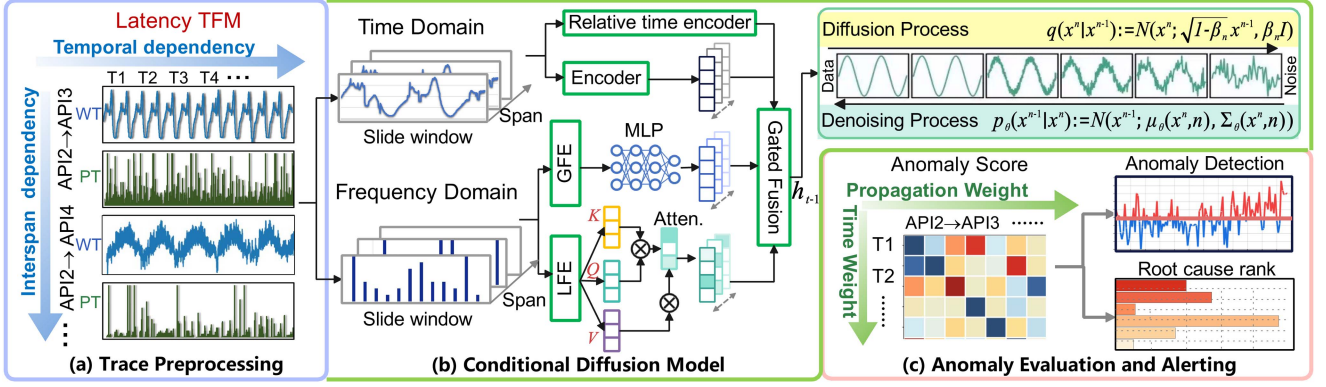


Fig. 5. The overview training and diagnosis framework of *TraceDiff*. (a) trace preprocessing: generate TFM to extract the temporal and interspan features from the trace; (b) conditional diffusion model: represent TFM as high-dimensional embeddings, and forecast the trace latency auto-regressively; (c) anomaly evaluation and alerting: compute abnormal scores of the alerting subjects and alert actionable anomalies.

approximate posterior is denoted as:

$$q(\mathbf{x}^{1:N}|\mathbf{x}^0) = \prod_{n=1}^N q(\mathbf{x}^n|\mathbf{x}^{n-1}). \quad (2)$$

where $\mathbf{x}^0$ represents the initial, uncorrupted observation sample, and I denotes the identity matrix of the same dimension. Each step of the (2) is:

$$q(\mathbf{x}^n|\mathbf{x}^{n-1}) := \mathcal{N}(\mathbf{x}^n; \sqrt{1 - \beta_n}\mathbf{x}^{n-1}, \beta_n \mathbf{I}) \quad (3)$$

Using a predetermined schedule of increasing variances, denoted as $\beta_1, \dots, \beta_N$, where each variance value is within the range of $\beta_n \in (0, 1)$. The joint distribution $p_\theta(\mathbf{x}^{0:N})$ is referred to as the reverse process, and N denotes the diffusion length. It is defined as a Markov chain with Gaussian transitions that have been learned. The process starts with $p(\mathbf{x}^N) = \mathcal{N}(\mathbf{x}^N; \mathbf{0}, \mathbf{I})$, and we can restore each $\mathbf{x}^0$ with learned Gaussian transitions follows this pattern:

$$p_\theta(\mathbf{x}^{n-1}|\mathbf{x}^n) := \mathcal{N}(\mathbf{x}^{n-1}; \mu_\theta(\mathbf{x}^n, n), \Sigma_\theta(\mathbf{x}^n, n)) \quad (4)$$

where parameters θ are learnable and shared; $\mu_\theta: \mathbb{R}^m \times \mathbb{N} \rightarrow \mathbb{R}^m$ and $\Sigma_\theta: \mathbb{R}^m \times \mathbb{N} \rightarrow \mathbb{R}^+$ are functions of the corrupted input $\mathbf{x}^n$ and the forward step's index $n \in \mathbb{N}_+$. Thus, the joint distribution is:

$$p_\theta(\mathbf{x}^0, \dots, \mathbf{x}^{n-1}|\mathbf{x}^N) := p(\mathbf{x}^N) \prod_{n=1}^N p_\theta(\mathbf{x}^{n-1}|\mathbf{x}^n). \quad (5)$$

The training parameters are trained to match the data distribution $q_\mathcal{X}(\mathbf{x}^0)$ by minimizing the KL-divergence between distributions p and q and reducing the negative log-likelihood using a variational bound employing Jensen's inequality.

$$\min_{\theta} \mathcal{L} := \min_{\theta} \mathbb{E}_{\mathbf{x}^0 \sim q(\mathbf{x}^0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t} \|\epsilon - \epsilon_\theta(\mathbf{x}^n, n)\|_2^2, \quad (6)$$

where ϵ indicates the noise that is introduced to the input during the forward process. ϵ_θ refers to a model that may be trained, which takes $\mathbf{x}^n = \sqrt{\bar{\alpha}_n}\mathbf{x}^0 + (1 - \bar{\alpha}_n)\epsilon$. Further, it is proved in [21] that $\mu_\theta(\mathbf{x}^n, n)$ and $\Sigma_\theta(\mathbf{x}^n, n)$ in (4) can be written as:

$$\mu_\theta(\mathbf{x}^n, n) := \frac{1}{\sqrt{\bar{\alpha}_n}} \left(x_n - \frac{\beta_n}{\sqrt{1 - \bar{\alpha}_n}} \epsilon_\theta(\mathbf{x}^n, n) \right),$$

$$\Sigma_\theta(\mathbf{x}^n, n) := \sigma_n^2 \mathbf{I} \quad (7)$$

where $\tilde{\beta} := \frac{1 - \alpha_{n-1}}{1 - \alpha_n} \beta_n$ if $n > 1$; $\tilde{\beta} := \beta_1$ if $n = 1$.

Therefore, it is evident that ϵ_θ is a network designed to forecast Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This network is trained using the mean squared error (MSE) loss, which measures the discrepancy between the actual noise and the expected noise. To generate samples from the reverse process $\mathbf{x}^{n-1} \sim p_\theta(\mathbf{x}^{n-1}|\mathbf{x}^n)$ after training, we can use the following approach to forecast the level of noise for any phase of the procedure and calculate it.

$$\mathbf{x}^{n-1} = \frac{1}{\sqrt{\bar{\alpha}_n}} \left(\mathbf{x}^n - \frac{\beta_n}{\sqrt{1 - \bar{\alpha}_n}} \epsilon_\theta(\mathbf{x}^n, n) \right) + \sqrt{\sigma_\theta} \mathbf{z} \quad (8)$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $n = N, \dots, 2$ and $\mathbf{z} = \mathbf{0}$ when $n = 1$. The whole sampling process for $\mathbf{x}^0$, initiated with a white noise sample $\mathbf{x}^n$, exhibits similarities to langevin dynamics. In this procedure, we sample from the distribution that is most affected by noise and gradually decrease the magnitude of the noise scale until we attain the smallest value.

2) *Conditioning Construction*: This section investigates advanced conditioning approaches, focusing on frequency-aware conditioning and temporal embeddings. It demonstrates how incorporating frequency information enhances the model's capability to identify dynamic latency patterns, and how temporal embeddings effectively capture the temporal sparsity present in latency.

Frequency-aware conditioning: A condition extraction block that incorporates time feature extraction, global frequency encoder (GFE), and local frequency encoder (LFE).

GFE: The GFE module (Fig. 5(b)) extracts global frequency information using the FFT transformation ($\mathcal{F}$). However, not all frequency data is useful; frequencies resulting from noise and anomalies in the time series data appear as long tails in the frequency domain. To address this, we employ a linear layer after the FFT to filter out the relevant frequency information that accurately represents the current window pattern. Additionally, we incorporate a dropout layer, following the approach used in Fedformer [28], to enhance the model's ability to learn missing frequency information. $f_{global} \in \mathbb{R}^{1 \times d}$ is calculated as $f_{global} =$

Dropout(Dense($\mathcal{F}(\mathbf{x})$)), where d is the embedding dimension of the global frequency information and $\mathcal{F}$ means FFT.

LFE: In the analysis of WT and PT data, we observe that the statistical properties change over time. Local frequency feature extraction highlights specific segments of a time series, capturing short-term patterns crucial for detecting anomalies that may not be visible in broader analyses. Identifying these anomalies is vital for effective anomaly detection. The attention mechanism effectively processes dependencies between time steps, allowing it to highlight crucial points in time series data, especially where anomalies occur. By combining local frequency features with attention mechanisms, the model better understands complex patterns and improves sensitivity to anomalies, enhancing anomaly detection accuracy. As illustrated in Fig. 5(b), the LFE module slides the window x to create smaller windows. The Fast Fourier Transform (FFT) is then applied to each window. The most recent small window serves as the query $\mathbf{x}_{sw}$ = SlidingWindow, while the others act as keys K and values V for attention. A linear layer helps the model extract key local frequency information, and dropout is used to improve the model's reconstruction of this information, similar to GFE. $f_{local} \in \mathbb{R}^{1 \times d}$ in LFE is given by (9), where d is the embedding dimension of the local frequency information.

$$K, V = \text{Dense}(\mathcal{F}(\mathbf{x}_{sw})), Q = \text{Select}(\text{Dense}(\mathcal{F}(\mathbf{x}_{sw})))$$

$$f_{local} = \text{Dropout}(\text{FeedForward}((\sigma(Q \cdot K^\top) \cdot V))) \quad (9)$$

Relative Time Embedding: We introduce a relative time embedding technique to address irregular time intervals. This technique captures the relative temporal relationships between time series elements through multi-resolution embeddings (days, hours, minutes). Specifically, the comprehensive embedding of a time series element is represented as:

$$Te(t) = E_D(t^d) + E_H(t^h) + E_M(t^m) \quad (10)$$

where days t^d , hours t^h , and minutes t^m can be calculated from slide window start time t , respectively. $E_D = [\sin(\frac{2\pi t^d}{365}), \cos(\frac{2\pi t^d}{365})]$, $E_H = [\sin(\frac{2\pi t^h}{24}), \cos(\frac{2\pi t^h}{24})]$ and $E_M = [\sin(\frac{2\pi t^m}{60}), \cos(\frac{2\pi t^m}{60})]$ represent the embedding functions for days, hours, and minutes. This embedding approach captures the periodic nature of time while preserving its relative aspects, allowing the model to better understand complex temporal relationships in trace data and enhancing its analytical performance.

Feature fusion: The integration of temporal, global spectral, local spectral, and relative temporal embedding features enables the generation of more precise hidden state representations. Various neural network architectures, including temporal convolutional networks (TCNs) [29] and advanced embedding methodologies [30], can be effectively employed as encoders to capture comprehensive temporal characteristics. $f_{time} \in \mathbb{R}^{1 \times d}$. The four features mentioned above are combined into a unified feature vector $\mathbf{h}_t = \text{fusion}(f_{time}, f_{global}, f_{local}, Te(t))$. This concatenation not only preserves the time domain information of the original time series but also enhances the frequency information by incorporating both global and local frequency features.

3) Trace Latency Model Based on Diffusion Architecture: This section enhances trace latency forecasting accuracy by incorporating conditions into naive denoising diffusion models. Denoising diffusion models are preferred over discriminative models for time series prediction due to their probabilistic generation approach. Their ability to model complex correlations effectively makes them highly suitable for trace anomaly detection, enabling the analysis of temporal dependencies and interspan correlations within trace data. Fig. 5(b) illustrates the latency model's procedure.

Training: Similar to a simplistic denoising diffusion model, the goal is to minimize the KL-divergence between p and q (6). The training objective, conditioned on $\mathbf{h}_t$, is to reduce the difference between $q_\theta(z|h)$ and $p_\theta(z|h)$. The training objective in (6) is reformulated and simplified based on certain conditions:

$$\min_{\theta} \mathcal{L} := \sum_{t=l}^{l+s} \min_{\theta} \mathbb{E}_{x_t^0, \epsilon, n} [\mathcal{L}_s(\epsilon - \epsilon_\theta(x_t^n, n, \mathbf{h}_{t-1}))] \quad (11)$$

where the context length is denoted by L , while the forecasting length is denoted by s .

Inference: Upon completing the training process, our goal is to predict future time steps for each span latency and evaluate these predictions against the test set. During training, we apply the GRU to the most recent window of the training set to compute the hidden state $\mathbf{h}_t$. We then use a sampling approach to generate a sample $\mathbf{x}_{T+1}^{n-1}$ for the next time step. This sample, along with the covariates c_{T+1} , is fed autoregressively into the GRU to produce the subsequent hidden state $\mathbf{h}_t$. This process is repeated iteratively until the desired forecast horizon is achieved.

$$x_{T+1}^{n-1} = \frac{1}{\sqrt{\alpha_n}} \left(x_{T+1}^n - \frac{\beta_n}{\sqrt{1 - \bar{\alpha}_n}} \epsilon_\theta(x_{T+1}^n, n, \mathbf{h}_t) \right) + \sqrt{\sigma_\theta} \mathbf{z}. \quad (12)$$

4) Accelerated Sampling Strategy: In standard diffusion probabilistic models, the reverse process entails iteratively denoising the latent variables from a high-noise state to the original signal over a complete sequence of N steps. While effective, this step-by-step traversal is computationally expensive and hinders real-time inference. To mitigate this overhead, we adopt the Accelerated Sampling Strategy [31]. This approach selects a strided subsequence of diffusion steps $\{\tau_1, \tau_2, \dots, \tau_M\}$ with $M \ll N$ to execute the denoising trajectory.

The accelerated sampling process skips intermediate steps while maintaining trajectory stability. For a selected subsequence of steps $\{\tau_1, \dots, \tau_M\}$, the update rule at step τ_m is formulated as follows:

$$\mathbf{x}_{\tau_{m-1}} = \underbrace{\sqrt{\bar{\alpha}_{\tau_{m-1}}} \left(\frac{\mathbf{x}_{\tau_m} - \sqrt{1 - \bar{\alpha}_{\tau_m}} \epsilon_\theta(\mathbf{x}_{\tau_m}, \tau_m)}{\sqrt{\bar{\alpha}_{\tau_m}}} \right)}_{\text{predicted } \mathbf{x}_0} + \underbrace{\sqrt{1 - \bar{\alpha}_{\tau_{m-1}} - \sigma_{\tau_m}^2} \cdot \epsilon_\theta(\mathbf{x}_{\tau_m}, \tau_m)}_{\text{direction to } \mathbf{x}_{\tau_{m-1}}} + \sigma_{\tau_m} \epsilon_{\tau_m}, \quad (13)$$

where $\bar{\alpha}_\tau$ represents the cumulative noise variance schedule defined in (6), $\epsilon_\theta(\mathbf{x}_{\tau_m}, \tau_m)$ is the noise predicted by the network conditioned on the step, and $\epsilon_{\tau_m} \sim \mathcal{N}(0, \mathbf{I})$ introduces stochasticity controlled by σ_{τ_m} .

This strategy significantly reduces the inference complexity from $O(N)$ to $O(M)$ by limiting the number of expensive neural network evaluations. Despite the reduced sampling resolution, the integration of precise step embeddings $\mathbf{e}(\tau_m)$ ensures that the model remains aware of the exact noise level, thereby preserving the fidelity of the generated latency baselines.

C. Anomaly Evaluation and Alerting

1) *Abnormal Score*: During the testing process, we utilize the Continuous Ranked Probability Score (CRPS) [32] as the abnormal score for each span. CRPS quantifies the degree of compatibility between a cumulative distribution function (CDF) F and an observation x and is calculated as $(F, x) = \int_{\mathbb{R}} (F(z) - \mathbb{I}\{x \leq z\})^2 dz$. The abnormal score of the trace can be calculated by summing the M spans of the specific trace data [33], resulting in $\hat{F}_{\text{sum}}(t)$ for each time point.

$$AbScore = \mathbb{E}_t \left[\text{CRPS} \left(\hat{F}_{\text{sum}}(t), \sum_{i=1}^D x_{i,t}^0 \right) \right]. \quad (14)$$

After obtaining the abnormal score, we incorporate anomaly trends using Mann-Kendall trend analysis [34] to calculate a more comprehensive indicator. Mann-Kendall trend analysis is a non-parametric statistical approach for detecting monotonic trends (i.e., continuous increases or decreases) in time series data. The TrendScore is calculated as follows:

$$TrendScore = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \text{sgn}(AbScore_j - AbScore_i) \quad (15)$$

where $AbScore_j$ and $AbScore_i$ are two data points in the AbScore sequence in (14) and $j > i$.

2) *Early Alert Assessment*: Early alerts are critical for identifying and responding to potential threats to microservices. Relying solely on real-time anomaly detection (e.g., failures, incidents, or interruptions) can lead to delays in certain scenarios [35]. Early alert assessment involves analyzing the relationship between current observations and the system's future state. We focus on the observed $TrendScore$, which measures the duration of deviations from normal behavior, indicating the likelihood of future anomalies.

3) *Real-Time Latency Anomaly Detection*: Anomalies are observations that deviate from expected patterns. In the context of trace analysis, an anomaly occurs when the actual value significantly diverges from the expected conditional distribution probability over a prediction horizon s . Specifically, our approach, TraceDiff, quantifies the severity of an anomaly using the $AbScore$. After computing the anomaly score for each trace, the next critical step is to determine a threshold that signals an anomaly event. For this purpose, we utilize a non-parametric, dynamic unsupervised thresholding method (NDT) as detailed in [36]. NDT creates the set e given a set of abnormal scores s of historical observations $e = \mu(s) + z\sigma(s)$, where z is an ordered

set of positive values. The threshold Th is calculated by

$$Th = \text{argmax}(e) = \frac{(\Delta\mu(s)/\mu(s)) + (\Delta\sigma(s)/\sigma(s))}{|s_a| + |E_{sea}^2|} \quad (16)$$

where $\Delta\mu(s) = \mu(s) - \mu(s < Th)$ and $\Delta\sigma(s) = \sigma(s) - \sigma(s < Th)$, NDT also considers preventing generating a large number of anomalous values $s_a = \{s > Th\}$ and anomalous sequences $|E_{sea}^2|$. NDT aims to identify a threshold that results in the most significant decrease in the average and variability of s by eliminating any values over the threshold.

4) *Root Cause Localization*: To improve root cause identification, we employ a comprehensive approach that includes abnormal scores, fault propagation modes, and timing factors. This approach assigns a root cause score to each API, indicating its likelihood of causing a failure. We enhance the identification of abnormal APIs as failure sources by examining the anomalies in each API and their immediate upstream and downstream counterparts. This involves creating a three-dimensional feature vector that captures the complexity of fault propagation $F_{\text{propagation}}^{(i)}$. Let Agg represent the approach used to aggregate the features of upstream or downstream instances. The sets of first-order downstream and upstream APIs are denoted as $V_{\text{down}}^{(i)}$ and $V_{\text{up}}^{(i)}$, respectively. The features aggregated from the instances in these sets are represented by $F_{\text{down}}^{(i)}$ and $F_{\text{up}}^{(i)}$.

$$F_{\text{propagation}}^{(i)} = \begin{pmatrix} F^{(i)} \\ F_{\text{down}}^{(i)} \\ F_{\text{up}}^{(i)} \end{pmatrix} = \begin{pmatrix} F^{(i)} \\ Agg \left(L^{(j)}, \forall j \in V_{\text{down}}^{(i)} \right) \\ Agg \left(L^{(k)}, \forall k \in V_{\text{up}}^{(i)} \right) \end{pmatrix} \quad (17)$$

Additionally, we acknowledge that various time intervals impact root cause determination differently. To address this, we apply a different weight to each time interval, allowing for a weighted average to signify their weight W_{time} . Finally, we derive a root cause score $RCS^{(i)}$ for each API i by combining the $AbScore$ from anomaly detection with the assigned importance weights.

$$RCS^{(i)} = AbScore \cdot F_{\text{propagation}}^{(i)} \cdot W_{\text{time}} \quad (18)$$

To identify the root cause, we calculated the root cause scores for each API. TraceDiff selects the top k scores with the highest RCS value. Finally, TraceDiff identifies the first k abnormal components as potential root causes.

V. EVALUATION

In this section, we focus on the evaluation to answer the following research questions:

- *RQ1*: How does TraceDiff perform in accurately detecting anomalies?
- *RQ2*: How can TraceDiff ensure that the latency distribution is precisely simulated with a high degree of granularity?
- *RQ3*: How interpretable is the failure diagnosis?
- *RQ4*: How does each component of TraceDiff contribute?
- *RQ5*: How do the hyperparameters of TraceDiff impact its performance?

TABLE I
BASIC INFORMATION ABOUT THREE DATASETS

Dataset	Traces	Spans	PT anomalies	WT anomalies
TrainTicket	24,637	236,010	3,781	3,380
SocialNetwork	44,065	170,412	2,349	2,012
Production System	1,506,723	240,348,982	21,924	14,501

A. Experiment Setting

1) *Dataset*: We conducted experiments using two widely recognized benchmarks: open-source microservice systems and a production microservice system. For the open-source systems, we selected TrainTicket (TT) [24] and SocialNetwork (SN) [25], two of the largest open-source microservice systems frequently used in prior research [7], [9], [15]. Following established approaches, we injected application bugs, CPU exhaustion, and network congestion into these systems and collected data using open-source monitoring tools, with traces sent to a Jaeger collector [37]. Performance was evaluated at three levels: microservice, container, and API. For the Production Microservice System (PMS), we collaborated with an Internet company serving over two million users. The dataset included 1,506,723 traces, with 36,425 (2.42%) affected by faults. Developers provided 22 faults across five categories: CPU exhaustion, memory exhaustion, host network errors, container network errors, and database failures. Table I summarizes the trace and span counts and the distribution of PT and WT anomalies.

While our evaluation primarily utilizes performance fault injections (e.g., CPU exhaustion, network congestion), these operational anomalies are fundamentally similar to specific security threats at the trace latency level. For instance, the CPU exhaustion scenarios evaluated in our datasets exhibit similar latency characteristics to Algorithmic Complexity Attacks (e.g., ReDoS) or resource hijacking (Cryptojacking), manifesting as severe PT anomalies. Similarly, the network congestion injections show similar anomaly characteristics to the queue saturation effects of Application-Layer DDoS attacks, triggering WT anomalies. Consequently, TraceDiff's state-of-the-art performance on these benchmark datasets directly translates to its efficacy as a symptom-based detector for availability-targeting security attacks.

2) *Evaluation Metrics*: To evaluate the accuracy of anomaly detection, following the common practice in anomaly detection [8], [9], [11], [14], we use Precision = $\frac{TP}{TP+FP}$, Recall = $\frac{TP}{TP+FN}$, and F1-score = $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ to evaluate the effectiveness of anomaly methods based on TP (True Positive), FP (False Positive), and FN (False Negative). We used Top-k accuracy (A@k), which refers to the probability that the root causes are included in the top-k results to evaluate the effectiveness of root cause location following the existing work [11], [15], [27]; we chose $k = 1, 2, 3$.

3) *Baselines*: To assess the anomaly detection performance, we compared TraceDiff with representative approaches categorized into two main groups: *statistic-based* and *model-based*. To maintain methodological clarity, the model-based approaches

are further subdivided into *VAE-based* and *diffusion-based* architectures.

Statistic-based approaches model latency distributions statistically based on structural features. While efficient, their coarse-grained grouping can limit modeling accuracy:

- *TraceRank* [11] uses spectrum analysis and PageRank-based random walks.
- *TraceRCA* [27] filters effective metrics based on anomaly severity changes.
- *TraceGra* [14] uses GNNs and LSTMs to extract temporal features.

Model-based approaches employ deep learning to model complex trace features. We evaluate two sub-categories:

1) VAE-based methods [8], [9], [15]:

- *TraceAnomaly* [9] applies posterior flow to a VAE using service trace vectors.
- *TraceVAE* [8] introduces a dual-variable graph VAE for trace latency and structure.
- *TraceSieve* [15] leverages VGAE and graph similarity analysis.

2) Diffusion-based methods:

- *Diffusion-TS* [38] disentangles trend, seasonal, and residual components in the time domain.
- *FourierDiffusion* [39] performs the diffusion process entirely in the frequency domain.
- *FCDM* [40] fuses both domains via an adaptive spectral-energy split for frequency-specific noise decay.

4) *Experimental Detail*: TraceDiff is implemented in Python 3.7.12 using PyTorch 1.8.0 as the primary deep learning framework. Experiments were conducted on a server equipped with two 32 T Intel(R) Xeon(R) Gold 5218 CPUs @ 2.30 GHz, one NVIDIA(R) Tesla(R) V100S GPU, and 188 GB of RAM. For the open-source benchmarks, the following parameters were configured: a learning rate of 0.001, condition length of 100, 8 residual layers, and hidden size of 100.

B. RQ1: Baseline Comparison

Table II presents the mean and standard deviation of the F1-scores across three datasets, evaluated under four scenarios: component latency (anomalies within a specific component or span), trace latency (anomalies across the entire trace), PT anomalies, and WT anomalies. Component latency anomalies, which are localized and occur on a smaller scale, are more challenging to detect than trace latency anomalies, requiring finer granularity in trace modeling. WT anomalies, often associated with network issues, exhibit clearer features compared to PT anomalies, which stem from varied causes, leading to higher detection accuracy for WT anomalies. The PMS dataset poses a greater detection challenge than TT and SN due to its larger data volume, increased complexity, and more intricate data correlations in a real-world environment. TraceDiff outperforms all baseline approaches, achieving the highest average F1-scores of 0.949, 0.959, and 0.917 across the three datasets.

The superior performance of TraceDiff can be attributed to three key innovations in the diffusion model architecture:

TABLE II

THE EFFECTIVENESS OF TRACEDIFF IN COMPARISON WITH ALL BASELINES ON THREE DATASETS OVER 10 INDEPENDENT RUNS. RESULTS ARE FORMATTED AS MEAN $\pm$ STANDARD DEVIATION. BOLD VALUES INDICATE THE BEST PERFORMANCE. SYMBOLS * DENOTE STATISTICALLY SIGNIFICANT IMPROVEMENTS OVER THE BEST-PERFORMING BASELINE AT $p < 0.05$ LEVEL.

Dataset	Approach	Anomalies Level		Anomalies Type	
		Component Latency	Trace Latency	PT Latency	WT Latency
TT	TraceRank	0.707 $\pm$ 0.012	0.744 $\pm$ 0.014	0.727 $\pm$ 0.011	0.753 $\pm$ 0.014
	TraceRCA	0.757 $\pm$ 0.016	0.765 $\pm$ 0.014	0.738 $\pm$ 0.018	0.777 $\pm$ 0.013
	TraceGra	0.752 $\pm$ 0.011	0.800 $\pm$ 0.013	0.754 $\pm$ 0.016	0.815 $\pm$ 0.012
	TraceAnomaly	0.768 $\pm$ 0.009	0.814 $\pm$ 0.010	0.772 $\pm$ 0.008	0.821 $\pm$ 0.011
	TraceVAE	0.835 $\pm$ 0.008	0.850 $\pm$ 0.009	0.841 $\pm$ 0.010	0.871 $\pm$ 0.008
	TraceSieve	0.900 $\pm$ 0.005	0.920 $\pm$ 0.006	0.886 $\pm$ 0.004	0.925 $\pm$ 0.007
	Diffusion-TS	0.848 $\pm$ 0.012	0.861 $\pm$ 0.010	0.838 $\pm$ 0.013	0.863 $\pm$ 0.009
	Fourier Diffusion	0.864 $\pm$ 0.009	0.900 $\pm$ 0.008	0.896 $\pm$ 0.010	0.913 $\pm$ 0.007
	FCDM	0.890 $\pm$ 0.006	0.922 $\pm$ 0.003	0.902 $\pm$ 0.008	0.919 $\pm$ 0.004
	TraceDiff (Ours)	0.929 $\pm$ 0.001*	0.960 $\pm$ 0.000*	0.945 $\pm$ 0.002*	0.961 $\pm$ 0.003*
SN	TraceRank	0.726 $\pm$ 0.014	0.773 $\pm$ 0.016	0.750 $\pm$ 0.015	0.781 $\pm$ 0.013
	TraceRCA	0.799 $\pm$ 0.013	0.807 $\pm$ 0.012	0.799 $\pm$ 0.014	0.817 $\pm$ 0.010
	TraceGra	0.818 $\pm$ 0.011	0.825 $\pm$ 0.010	0.815 $\pm$ 0.013	0.833 $\pm$ 0.011
	TraceAnomaly	0.841 $\pm$ 0.011	0.860 $\pm$ 0.009	0.832 $\pm$ 0.010	0.865 $\pm$ 0.008
	TraceVAE	0.919 $\pm$ 0.009	0.916 $\pm$ 0.007	0.913 $\pm$ 0.008	0.928 $\pm$ 0.006
	TraceSieve	0.871 $\pm$ 0.004	0.954 $\pm$ 0.005	0.943 $\pm$ 0.006	0.965 $\pm$ 0.004
	Diffusion-TS	0.837 $\pm$ 0.010	0.859 $\pm$ 0.011	0.847 $\pm$ 0.013	0.899 $\pm$ 0.009
	Fourier Diffusion	0.860 $\pm$ 0.011	0.864 $\pm$ 0.010	0.889 $\pm$ 0.012	0.913 $\pm$ 0.008
	FCDM	0.913 $\pm$ 0.005	0.958 $\pm$ 0.002	0.932 $\pm$ 0.007	0.967 $\pm$ 0.004
	TraceDiff (Ours)	0.950 $\pm$ 0.003*	0.963 $\pm$ 0.001*	0.945 $\pm$ 0.005*	0.977 $\pm$ 0.001*
PMS	TraceRank	0.566 $\pm$ 0.021	0.609 $\pm$ 0.018	0.598 $\pm$ 0.022	0.620 $\pm$ 0.019
	TraceRCA	0.679 $\pm$ 0.018	0.738 $\pm$ 0.019	0.712 $\pm$ 0.021	0.751 $\pm$ 0.017
	TraceGra	0.717 $\pm$ 0.016	0.776 $\pm$ 0.014	0.758 $\pm$ 0.018	0.797 $\pm$ 0.015
	TraceAnomaly	0.805 $\pm$ 0.015	0.824 $\pm$ 0.012	0.810 $\pm$ 0.014	0.831 $\pm$ 0.011
	TraceVAE	0.871 $\pm$ 0.010	0.902 $\pm$ 0.011	0.867 $\pm$ 0.013	0.910 $\pm$ 0.009
	TraceSieve	0.833 $\pm$ 0.008	0.860 $\pm$ 0.007	0.835 $\pm$ 0.009	0.875 $\pm$ 0.008
	Diffusion-TS	0.810 $\pm$ 0.013	0.840 $\pm$ 0.012	0.818 $\pm$ 0.015	0.840 $\pm$ 0.011
	Fourier Diffusion	0.842 $\pm$ 0.011	0.856 $\pm$ 0.010	0.830 $\pm$ 0.013	0.847 $\pm$ 0.009
	FCDM	0.877 $\pm$ 0.007	0.903 $\pm$ 0.008	0.861 $\pm$ 0.006	0.911 $\pm$ 0.007
	TraceDiff (Ours)	0.907 $\pm$ 0.003*	0.927 $\pm$ 0.004*	0.907 $\pm$ 0.001*	0.927 $\pm$ 0.004*

- 1) *Enhanced Distribution and Correlation Modeling*: Unlike existing approaches (TraceRank and TraceRCA) that overlook temporal and inter-span dependencies, and VAE-based methods that assume Gaussian latent spaces, our diffusion model explicitly captures the intrinsic correlations within traces through its multi-step denoising process. This enables more accurate detection of component latency anomalies, especially for complex, multi-modal latency patterns (evidenced by improved performance on PT anomalies, which exhibit varied patterns).
- 2) *Multi-Scale Frequency Analysis with Conditional Generation*: Through our novel dual-encoder (GFE and LFE) architecture, TraceDiff effectively captures both long-term trends and short-term fluctuations in latency patterns. Moreover, diffusion models naturally accommodate conditional inputs throughout the generation process, allowing seamless integration of these frequency features and temporal embeddings. This multi-scale, condition-aware representation enables optimal performance even for complex, interconnected traces in production microservice systems (**Solutions to challenge 1**).

- 3) *Robust Training Framework with Sparsity Tolerance*: Compared to VAE-based methods, our diffusion model demonstrates superior training stability through stochastic modeling via a noise/denoising process. The denoising process performs implicit interpolation across temporal gaps, making it less sensitive to sparse observations compared to Transformer-based models that rely on continuous context. This is further enhanced by the incorporation of relative time embedding as conditional inputs, enabling effective handling of temporal irregularities (**Solutions to challenge 2**). These advantages are supported by strong performance on the PMS dataset with high sparsity.

Regarding efficiency and large-scale deployment feasibility, we evaluated the training and testing time overhead of TraceDiff against baselines across the three datasets (Fig. 6). While statistical approaches require significantly less time, their inability to model complex inter-span correlations leads to unacceptable accuracy trade-offs in production. Among the model-based approaches, TraceDiff demonstrates a highly practical accuracy-efficiency balance. On the massive PMS dataset (1.5×10^6 traces), TraceDiff requires approximately 157 minutes for offline training. Given the sheer volume of production data, this training

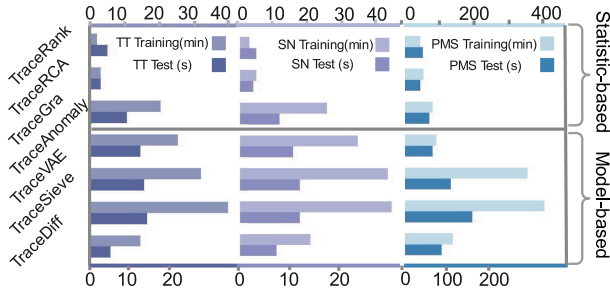


Fig. 6. Comparison of efficiency between TraceDiff and different approaches.

TABLE III
PERFORMANCE OF ROOT CAUSE LOCALIZATION. ‘N/A’ AND ‘-’ DENOTE
BASELINES INCOMPATIBLE WITH TOP- k EVALUATIONS.

Approach	TT			SN			PMS		
	A@1	A@2	A@3	A@1	A@2	A@3	A@1	A@2	A@3
TraceRank	0.71	0.74	0.82	0.79	0.86	0.92	0.62	0.68	0.75
TraceRCA	0.67	0.72	0.79	0.82	0.84	0.89	0.72	0.82	0.86
TraceGra	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
TraceAnomaly	0.61	-	-	0.78	-	-	0.74	-	-
TraceVAE	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
TraceSieve	0.79	0.82	0.89	0.83	0.89	0.91	0.75	0.79	0.84
w/ VAE	0.80	0.81	0.89	0.83	0.88	0.92	0.78	0.86	0.90
w/o LFE & GFE	0.77	0.80	0.91	0.84	0.90	0.93	0.76	0.83	0.87
w/o Time_W	0.85	0.88	0.92	0.87	0.92	0.95	0.83	0.88	0.90
w/o Prop_W	0.75	0.77	0.83	0.80	0.84	0.88	0.71	0.79	0.83
TraceDiff	0.91	0.94	0.96	0.92	0.96	0.97	0.87	0.92	0.94

overhead is highly manageable for standard periodic retraining pipelines (e.g., daily or weekly model updates) in industrial settings. More importantly, for real-time online deployment, inference latency is the primary bottleneck. As shown in Fig. 6, TraceDiff achieves a testing time of just 71 seconds on the PMS dataset. By keeping the testing time strictly bounded, TraceDiff comfortably satisfies the strict low-latency requirements for real-time anomaly alerting in large-scale microservice environments, avoiding the severe computational bottlenecks.

Regarding root cause localization, TraceDiff consistently outperforms all baselines across the three datasets (Table III). Note that TraceGra and TraceVAE are marked ‘N/A’ as they only support trace-level classification, lacking span-level ranking mechanisms. Similarly, TraceAnomaly lacks A@2 and A@3 because it solely outputs a Top-1 prediction. For instance, on the highly complex PMS dataset, TraceDiff achieves an impressive A@3 of 0.94 and an A@1 of 0.87, significantly surpassing the best-performing model-based baselines. Its robust performance, particularly in A@1 (Top-1 accuracy), highlights TraceDiff’s superior capability to precisely pinpoint the exact anomalous component, rather than merely narrowing down a general fault area, even in large-scale, intricate microservice environments.

C. RQ2: Effectiveness of Fine-Grained Latency Detection

The failures collected in the PMS dataset included severe anomalies (e.g., significant performance drops across multiple traces due to severe network failures) and subtle anomalies

TABLE IV
THE EFFECTIVENESS OF LATENCY MODELING IN COMPARISON WITH
DIFFERENT APPROACHES

Approach	Component Latency anomaly type		
	Temporal	Interspan	Interspan-temporal
TraceRank	0.673	0.365	0.787
TraceRCA	0.684	0.412	0.763
TraceGra	0.737	0.523	0.809
TraceAnomaly	0.824	0.781	0.853
TraceVAE	0.831	0.754	0.842
TraceSieve	0.838	0.768	0.866
Diffusion-TS	0.821	0.744	0.833
FourierDiffusion	0.829	0.758	0.849
FCDM	0.844	0.774	0.861
w/ VAE	0.745	0.643	0.811
w/o LFE&GFE	0.843	0.792	0.872
w/o LFE	0.878	0.854	0.899
w/o GFE	0.869	0.849	0.903
w/o Attention	0.882	0.859	0.912
TraceDiff	0.925	0.918	0.937

(e.g., minor fluctuations in response time for specific interaction pairs caused by burst congestion). These observations demonstrate TraceDiff’s effectiveness in detecting anomalies in a production environment. We categorize latency anomalies into three types: temporal anomalies, interspan anomalies, and interspan-temporal anomalies in Table IV.

Temporal Anomalies: Deep learning-based approaches, including VAE-based methods (TraceAnomaly, TraceVAE) and general time-series diffusion models (Diffusion-TS, FourierDiffusion, FCDM), exhibit higher accuracy in detecting temporal anomalies compared to statistic-based approaches (TraceRank, TraceRCA). Specifically, FCDM achieves a competitive F1-score of 0.844 by leveraging frequency information. However, TraceDiff achieves the best F1 score of **0.925**, significantly outperforming the baselines. This underscores the robustness of our conditional diffusion model in capturing dynamic temporal patterns and local fluctuations.

Interspan Anomalies: This category presents the most significant challenge. Statistic-based approaches fail to model inter-span correlations, resulting in low F1-scores (0.365–0.412). Notably, even state-of-the-art general diffusion models (Diffusion-TS, FourierDiffusion, FCDM) struggle in this scenario, achieving F1-scores between 0.744 and 0.774. This limitation arises because they treat traces as flattened time series, ignoring the intrinsic caller-callee dependency structure. In contrast, TraceDiff achieves a remarkable F1-score of **0.918**. By leveraging relative time embeddings and structure-aware conditioning, TraceDiff effectively captures anomalies in highly correlated data, identifying concurrent changes in related spans that generic models miss.

Interspan-Temporal Anomalies: Most algorithms achieve higher F1-scores for this category, as anomalies are easier to detect from either a temporal or trace perspective. Nevertheless, TraceDiff consistently outperforms all baselines, achieving an F1-score of **0.937**, while the second-best method (TraceSieve) reaches only 0.866. This demonstrates TraceDiff’s superior capability in fusing interspan correlations with temporal dependencies to detect complex hybrid anomalies.

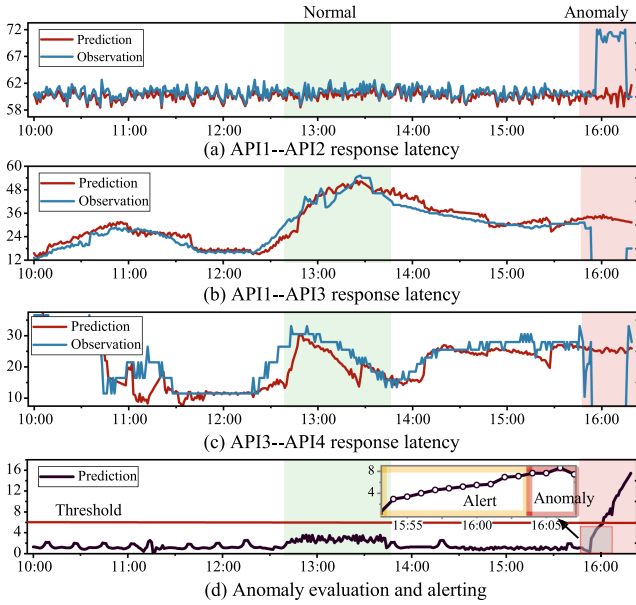


Fig. 7. Successful cases of TraceDiff in forecasting span latency and anomaly evaluation.

D. RQ3: Interpretability of Failure Diagnosis

1) *Explainable Deviations and Proactive Alerting*: To evaluate TraceDiff’s interpretability, we conducted a case study using a subset of the PMS dataset. Fig. 7(a), (b), and (c) display span latency predictions, with the blue line representing actual observations and the red line showing TraceDiff’s predictions. Fig. 7(d) presents the abnormal score for the trace, along with alert and anomaly regions. In Fig. 7(b) and (c), two peaks labeled as “normal” would typically trigger alerts in TraceRank and TraceRCA. However, our method considers temporal and span dependencies, resulting in lower abnormal scores during these periods compared to the clearly “abnormal” section highlighted in red. Indeed, the “normal” spikes are caused by typical increases in user requests, while the abnormal segment indicates a physical machine failure, leading to irregular patterns. The anomaly evaluation and alerting phase identify abnormal trace characteristics by analyzing Abnormal Scores and Trend Scores, assessing anomalies’ severity and persistence. As shown in Fig. 7(d), TraceDiff effectively determines when to issue alerts and alarms. Experimental results demonstrate that TraceDiff can generate alerts 10 to 25 time points in advance of anomalies, enabling early intervention to prevent significant failures.

2) *Interpretable Diagnosis via Spectral Attention*: Additionally, we designed a controlled experiment to examine the behavior of the Local Frequency Encoder (LFE) module in distinguishing normal operational patterns from genuine anomalies. We selected two representative scenarios from the production dataset: **Scenario A (Normal)**: Recurring network jitter patterns that occur periodically during normal operations. **Scenario B (Anomaly)**: Sudden burst patterns caused by a database deadlock that had not occurred previously. For each scenario, we extracted the frequency domain representations using FFT and computed attention weights between the current window and historical windows. As shown in Fig. 8, the frequency-aware

attention mechanism exhibits distinctly different behaviors for normal and abnormal patterns: The frequency spectrum of the current pattern shows characteristic peaks corresponding to known periodic jitter. The attention mechanism successfully retrieves similar historical patterns with high similarity scores. The frequency spectrum exhibits unusual patterns not seen in historical data. The attention mechanism fails to find similar historical patterns, reflecting the absence of matching patterns.

3) *Visualizing Root Cause Localization*: Beyond simply detecting an anomaly, an operator must know where to act. Complex microservice dependencies often cause cascading failures, making root cause localization difficult. We explicitly illustrate how TraceDiff resolves this in Fig. 9 by synthesizing three key dimensions: the base Abnormal Score, Fault Propagation Weight, and Time Weight.

- *Example 1*: When the root cause is isolated (API 2), the base Abnormal Score clearly identifies the origin.
- *Example 2*: When an upstream root cause (API 2) heavily impacts a downstream service (API 3), applying the propagation weight explicitly untangles this dependency, correcting the focus back to the true upstream origin.
- *Example 3*: When asynchronous temporal delays obscure the source, incorporating the Time Weight effectively filters out temporal noise to successfully isolate the delayed root cause (API 3).

This scoring provides a transparent, step-by-step localization rationale. By explicitly mapping the impact scope and the definitive root cause, TraceDiff translates complex topological alerts into clear directives for rapid system mitigation.

E. RQ4: Ablation Study

To deeply understand the structural contributions of our proposed framework, we decompose the ablation study into two progressive phases corresponding to TraceDiff’s workflow: Anomaly Detection and Root Cause Localization.

1) *Impact on Anomaly Detection*: In this phase, we evaluate how the core representation learning modules (i.e., the diffusion model, frequency encoders, and attention mechanism) affect TraceDiff’s ability to accurately identify abnormal traces (measured by F1-scores).

- *Diffusion Model versus VAE*: Replacing the diffusion backbone with a VAE (w/ VAE) causes a severe performance degradation across all anomaly types (Table IV). Most notably, the F1-score for interspan anomalies plummets from 0.918 (TraceDiff) to merely 0.643 (w/ VAE). Furthermore, state-of-the-art VAE baselines (e.g., TraceSieve and TraceAnomaly) struggle to capture complex structural correlations, peaking at 0.781 for interspan anomalies. This explicitly demonstrates that the diffusion model’s iterative denoising process is significantly more robust at modeling intricate temporal and interspan dependencies than traditional VAE architectures.
- *Frequency-aware Conditions (LFE & GFE)*: We employ both GFE and LFE to capture complementary frequency information. Table IV shows that integrating these encoders significantly outperforms VAE-based approaches

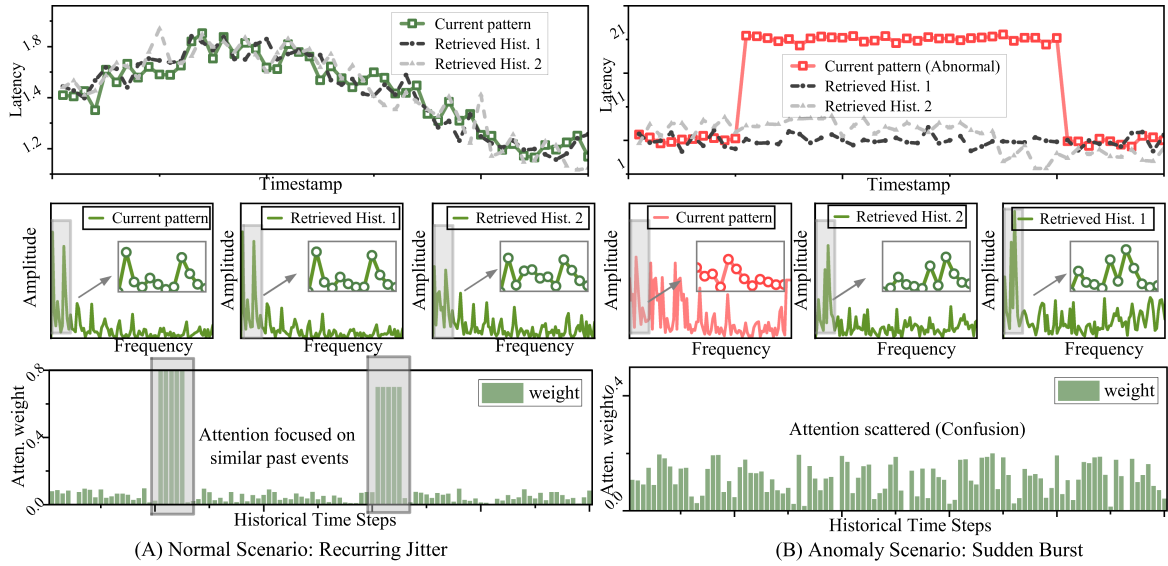


Fig. 8. Frequency-aware attention analysis for interpretable anomaly detection.

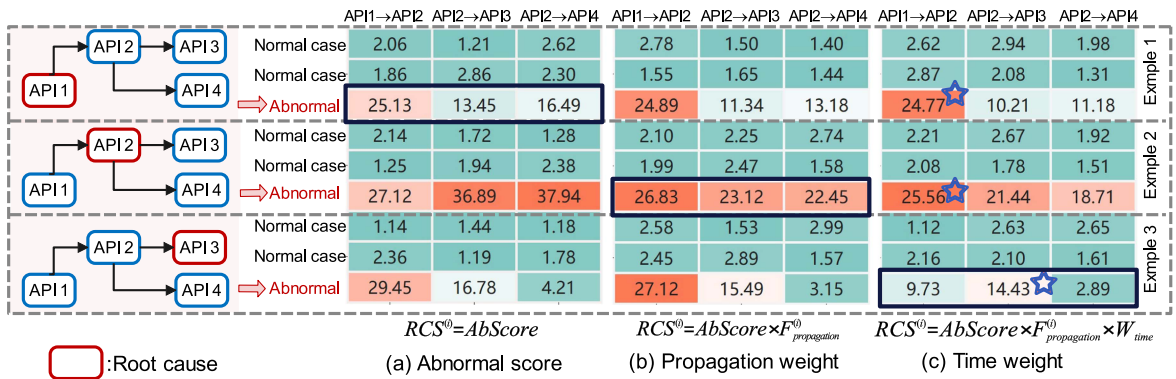


Fig. 9. The weight for root cause localization. The dashed boxes highlight three illustrative examples, while the solid black boxes denote the weights that contributed to identifying the root cause. The stars mark the root cause spans.

(e.g., TraceAnomaly, TraceSieve). While their extracted features exhibit partial overlap, the synergistic integration of global macro-trends and local frequency patterns proves critical for maximizing detection accuracy.

- **Attention Mechanism:** The LFE captures overall frequency features while emphasizing important local variations. Removing the attention mechanism led to a noticeable decline in detection performance. By dynamically evaluating the similarity between small temporal windows and adjusting their weights, the attention mechanism significantly enhances the model's sensitivity to localized latency spikes.

2) **Impact on Root Cause Localization:** Beyond merely detecting anomalies, the ultimate goal of TraceDiff is to pinpoint the exact faulty microservice. In this phase, we evaluate the structural contributions of both the feature extractors and the routing weights to the Top- k accuracy (Table III).

- **Topological Routing (Propagation Weight):** The Propagation Weight ($Prop_W$) emerges as the most decisive component for root cause localization. Its ablation causes the most severe performance plunge across all

datasets. This empirically confirms that mitigating fault cascading along the caller-callee topology is the ultimate bottleneck in microservice RCA; without it, the model is easily overwhelmed by symptomatic downstream alerts.

- **Feature Extraction (LFE & GFE):** Robust frequency-aware feature extraction acts as a strict prerequisite for accurate routing. Removing the frequency encoders (w/o LFE & GFE) causes a highly significant degradation (e.g., A@1 drops to 0.76 on PMS). If the base anomaly score ($AbScore$) is flawed due to poor feature extraction, even the best routing mechanism will fail to pinpoint the root cause.

F. RQ5: Hyperparameters Sensitivity

The length N of the diffusion model is a crucial hyperparameter that influences both the noise addition process and the recovery of the original data. A larger N enables the reverse process to better approximate a Gaussian distribution, enhancing parameterization accuracy. However, longer diffusion lengths

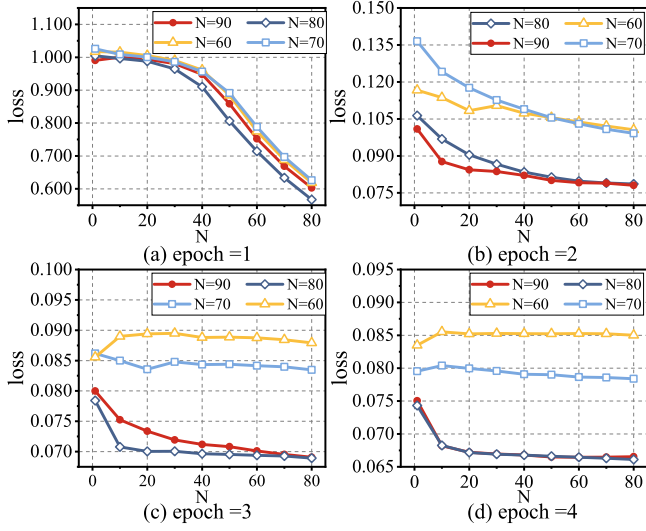


Fig. 10. Training loss of TraceDiff with varying diffusion lengths N on TT Dataset. The batch number of each epoch is 80, and the batch size of each batch is 64.

TABLE V
QUALITY VERSUS SPEED TRADE-OFF OF ACCELERATED SAMPLING ON THE PMS DATASET (TRAINED WITH $N = 80$)

Inference Steps (M)	Detection Quality (F1)		Inference Speed	
	Component Level	Trace Level	Time / Trace	Speedup
$M = 80$	0.909 ± 0.002	0.929 ± 0.003	42.4 ms	$1.0\times$
$M = 60$	0.908 ± 0.002	0.929 ± 0.003	21.3 ms	$2.0\times$
$M = 40$	0.908 ± 0.003	0.928 ± 0.002	10.8 ms	$3.9\times$
$M = 20$	0.907 ± 0.003	0.927 ± 0.004	5.4 ms	$7.8\times$
$M = 10$	0.812 ± 0.015	0.834 ± 0.018	2.8 ms	$15.1\times$

increase computational demands, as each step in the reverse process must be estimated and optimized. We evaluate how N affects prediction performance, where we record the train set loss of the TT dataset for different total diffusion process lengths $N = 60 - 90$ while keeping all other hyperparameters unchanged. The results are then plotted in Fig. 10 where an optimal value is achieved at $N = 80$ and larger levels are not beneficial if all else is fixed.

While the training diffusion length N determines the parameterization accuracy, the number of inference steps M directly dictates the real-time viability of TraceDiff. To substantiate our claims of computational efficiency, we conducted a trade-off analysis on the PMS dataset by fixing the trained model at $N = 80$ and varying the inference steps $M \in \{80, 60, 40, 20, 10\}$. As shown in Table V, executing the full reverse process ($M = 80$) yields the highest detection accuracy. By adopting the accelerated sampling strategy and reducing M to 20, TraceDiff achieves an $8\times$ inference speedup with negligible performance degradation (e.g., the Trace-level F1-score drops by merely 0.002). However, aggressively reducing M to 5 introduces severe discretization errors, leading to a significant drop in F1-score.

G. Qualitative Comparison of SOTA Paradigms

To consolidate our quantitative results, we qualitatively compare TraceDiff with representative paradigms regarding

their strengths, limitations, and expected real-world behaviors. **Statistic-based approaches** (e.g., TraceRank, TraceRCA) are highly efficient but rely on coarse-grained distributions that overlook complex inter-span dependencies. Thus, they effectively detect massive system failures but frequently miss subtle, localized bottlenecks. **VAE-based approaches** (e.g., TraceAnomaly, TraceSieve) are adept at capturing non-linear temporal features; however, their fixed Gaussian prior over-smooths multi-modal distributions. Being highly vulnerable to temporal sparsity, they often misclassify legitimate dynamic shifts as noise. **General diffusion-based approaches** (e.g., Diffusion-TS, FCDM) excel at modeling non-Gaussian distributions but treat traces as flattened time-series, ignoring caller-callee topologies. Consequently, they succeed in purely temporal anomaly detection but struggle significantly with inter-span structural anomalies. In contrast, **TraceDiff** integrates the Trajectory Feature Matrix (TFM) with frequency-aware diffusion to bridge temporal gaps and robustly model topological correlations. While multi-step denoising introduces computational overhead, our accelerated sampling mitigates this, ultimately ensuring SOTA accuracy and precise root cause localization in dynamic, sparse environments.

VI. COMPLEXITY ANALYSIS

In real-time microservice monitoring, computational efficiency is as critical as detection accuracy. This section systematically analyzes the time and space complexity of TraceDiff and compares it with state-of-the-art baselines. Let L denote the trace sequence length, d the embedding dimension, and N the standard number of diffusion steps.

The inference efficiency of TraceDiff is determined by two primary components: frequency feature extraction and the iterative diffusion backbone.

- *Frequency Encoding*: The Global and Local Frequency Encoders (GFE/LFE) rely on Fast Fourier Transform (FFT) operations. The complexity for processing a sequence of length L scales log-linearly as $O(L \log L \cdot d)$.
- *Iterative Denoising*: As detailed in Section IV-B4, we employ the *Accelerated Sampling Strategy* to optimize the reverse process. By reducing the number of diffusion iterations from N to a strided subset M (where $M \ll N$), the computational cost of the GRU-based backbone is significantly lowered. Since each GRU step incurs $O(L \cdot d^2)$, the total complexity for the diffusion process is $O(M \cdot L \cdot d^2)$.

Combining these components, the overall inference time complexity of TraceDiff is derived as:

$$\mathcal{T}_{\text{TraceDiff}} \approx O(M \cdot L \cdot d^2 + L \log L \cdot d) \quad (19)$$

We compare the scalability of TraceDiff against two dominant categories of baseline models:

- *Vs. Transformer-based Models*: Standard Transformer architectures rely on self-attention mechanisms that compute pairwise interactions between all time steps, resulting in a quadratic time complexity of $O(L^2 \cdot d)$. In contrast, TraceDiff maintains a linear complexity $O(L)$ regarding sequence length. This linear scaling provides a substantial

efficiency advantage when processing long traces common in complex microservice architectures.

- *Vs. VAE-based Models:* VAE-based approaches (e.g., TraceVAE [10]) typically operate with a single inference pass, yielding a complexity of $O(1 \cdot L \cdot d^2)$. While TraceDiff introduces a constant multiplier M (typically $M = 10$), this is a controlled trade-off designed to achieve superior generative robustness and modeling capacity (as demonstrated in Section V-F). Given that M is a small constant, TraceDiff preserves the same order of magnitude in efficiency while significantly outperforming VAEs in detecting complex anomalies.

VII. CASE STUDIES ON REAL-WORLD EFFECTIVENESS

To further demonstrate TraceDiff’s capability in handling both operational faults and security threats, we analyze two representative cases detected during our evaluation on the Production Microservice System (PMS). These cases empirically validate that our generative denoising mechanism effectively highlights, rather than obscures, critical anomalies.

A. Reliability Case: Software Bug Detection

The first case highlights TraceDiff’s efficacy in ensuring service reliability through the detection of software defects. During a routine monitoring interval, the model triggered a high-severity alert for a payment service API. Specifically, a significant Process Time (PT) anomaly was observed, where execution time escalated from a baseline of 45 ms to over 2000 ms, resulting in timeouts. Crucially, the diffusion model conditioned on healthy historical contexts accurately forecasted a low-latency baseline. The stark divergence between this generated prediction and the actual high-latency observation enabled precise anomaly pinpointing. Subsequent investigation revealed a logic bug in the exception handling mechanism; under rare input combinations, the service entered an unintended retry loop instead of failing fast, causing the thread to hang. TraceDiff’s granular span-level localization allowed the engineering team to rapidly identify the failing component and deploy a hotfix, thereby restoring system stability.

B. Security Case: Injection-Induced Latency

The second case addresses security threats and explicitly validates the robustness of our generative approach against adversarial patterns. In this instance, TraceDiff detected a persistent surge in Process Time within a search service, occurring despite relatively normal traffic volumes. TraceDiff’s denoising process driven by the learned distribution of normal behavior remained stable, continuing to generate a low-latency prediction consistent with normal operations. Consequently, the “attack signal” was not smoothed out; instead, the stability of the generated baseline maximized the deviation between the prediction and the observation. Security analysis later confirmed the root cause to be a malicious SQL injection attempt, where complex queries forced full-table database scans and exhausted CPU resources. The early alert enabled the security team to identify the source

TABLE VI
DETECTION PERFORMANCE ON THE GRADUAL EXHAUSTION (ADAPTIVE ATTACK) SCENARIO

Approach	F1-Score	Alert Trigger Mechanism
TraceAnomaly	0.48	Day 7 (Alerted only upon crash).
TraceDiff (w/o GFE)	0.75	Day 5 (Alerted during severe degradation).
TraceDiff (Ours)	0.90	Day 3 (Early alert triggered by GFE).

IP and block the malicious requests, effectively mitigating a potential availability attack.

C. Security Case: Adaptive Resource Exhaustion

As a symptom-based service availability and performance evaluator, the core objective of TraceDiff is fine-grained anomaly detection and root cause localization. From the perspective of distributed tracing, whether a system disruption is triggered by an operational fault (e.g., software bug, network failure) or a malicious security attack, the resulting structural deviations inherently manifest as identical trace-level symptoms: severe anomalies in WT and PT. Therefore, accurately detecting these equivalent operational faults inherently demonstrates TraceDiff’s robust defensive capability against corresponding security threats. We demonstrate this symptom-equivalence through an empirical security scenario.

Beyond sudden bursts, security experts are deeply concerned with adaptive or evasive attacks, where an attacker incrementally increases resource consumption over several days to successfully poison the baseline of unsupervised learning models. To empirically evaluate TraceDiff’s robustness against such advanced threats, we extracted a natural fault from the PMS dataset with exact structural equivalence: a gradual memory leak. A memory leak perfectly mimics a slow-rate adaptive resource exhaustion attack, causing a creeping, monotonic rise in PT of merely 2%–5% per day due to increasing Garbage Collection (GC) overhead. As shown in Table VI, traditional sliding-window models (like VAE-based TraceAnomaly) smoothly adapt to this slow drift, treating the poisoned data as the “new normal” until a catastrophic Out-Of-Memory (OOM) crash occurs on Day 7. TraceDiff mitigates this vulnerability through its Global Frequency Encoder (GFE). While the daily time-domain shift is hidden in background noise, this cumulative drift significantly alters the low-frequency energy distribution. TraceDiff detects this spectral distortion and triggers an early alert on Day 3, effectively resisting baseline poisoning.

VIII. DISCUSSION AND THREATS TO VALIDITY

In its operational deployment within the production microservice environment, TraceDiff demonstrated exceptional performance by identifying and reporting 141 actionable anomalies to engineering teams, achieving a remarkable 92% confirmation rate (130 verified cases). Notably, it identified a latent fault at the component level, involving a logical error in application code that was overlooked by existing approaches. TraceDiff’s analysis of interspan relationships correctly pinpointed the faulty API interface. Furthermore, TraceDiff’s proactive latency prediction capability provides early alerts, offering preventive

maintenance, reduced downtime, and optimized system performance.

The core hyperparameters of diffusion models, including diffusion length, noise scheduling, and learning rate, are relatively few, simplifying the training process. However, the computational demands are higher due to the necessity of forward and reverse propagation at each diffusion step. TraceDiff is less sensitive to core parameter variations, though the diffusion length involves a trade-off between prediction performance and computational cost. We recommend shorter diffusion lengths to minimize sample generation time for services requiring rapid anomaly detection, such as core service systems or real-time monitoring. Conversely, longer diffusion lengths may be preferable for tasks like offline data analysis or focused fault analysis.

IX. RELATED WORK

Anomaly detection in microservices ensures system stability, reliability, and operational efficiency [2], [41]. Current approaches primarily leverage three data sources: metrics, logs, and traces, which collectively provide comprehensive insights into microservice behavior [1], [3]. Consequently, anomaly detection in microservices can be categorized into three primary approaches: log-based [42], [43], [44], metric-based [4], [5], and trace-based detection [7], [8], [15], [26]. We specifically focus on trace-based anomaly detection research.

Historically, distributed systems captured interactions via runtime paths and client requests [7], [8], [11], [14], [45]. Today, trace-based failure diagnosis integrates structural and contextual features, with current research primarily categorized into two paradigms: statistic-based and model-based approaches [26].

Statistical-based approaches [6], [7], [11], [14], [27] employ structural similarity to group trace nodes and subsequently model latency distributions using statistical techniques. While computationally efficient, these approaches exhibit limitations in modeling accuracy due to their coarse-grained grouping strategies. This limitation stems from the inherent variability in component latency across different trace paths [7], [15]. For instance, database interactions may demonstrate significantly different write durations when processing image files compared to plain text. Although Traceark [7] incorporate fine-grained features, they still require engineering expertise for anomaly model refinement. Consequently, effective anomaly assessment necessitates unsupervised approaches that incorporate detailed trace structure information.

Model-based approaches [8], [9], [15], [45] have emerged as a promising direction, leveraging fine-grained trace structure embeddings and service behavior dynamics to achieve superior detection accuracy. Current deep learning approaches can be classified into two principal categories: Graph Neural Network (GNN)-based approaches and Variational Autoencoder (VAE)-based techniques. The GNN-based paradigm includes approaches such as DeepTraLog [46], which integrates trace and log data through GNNs for anomaly detection, and TraceCRL [45], which employs graph contrastive learning for trace anomaly detection and sampling. The VAE-based approaches encompass TraceVAE [8], which extends trace modeling with graphVAE, while GRAND [16] and TraceSeive [15] combine

Variational Autoencoders with Generative Adversarial Networks to enhance anomaly detection in execution traces while mitigating noise interference. Existing approaches struggle to manage temporal dynamics and sparse traces. Consequently, they fail to provide sufficient support for fine-grained fault diagnosis in microservice systems.

X. CONCLUSION

Anomaly detection based on traces plays a critical role in ensuring the quality and reliability of microservice systems. TraceDiff is the first approach that utilizes a novel conditional denoising diffusion model for accurately predicting trace latency and detecting anomalies. TraceDiff employs frequency-aware analysis technology, which is capable of capturing both global and local dynamic changes while accurately extracting the most critical frequency components, thereby effectively addressing the temporal dynamics of traces. Furthermore, TraceDiff leverages a diffusion model to facilitate more precise self-supervised modeling of the intricate temporal and interweaving correlations that are characteristic of traces, addressing the temporal sparsity of traces. TraceDiff specializes in capturing temporal dependencies and correlations between spans, enabling it to accurately characterize latency and effectively capture variations, fluctuations, and atypical patterns. Extensive experiments on public datasets demonstrate that TraceDiff outperforms state-of-the-art baselines in terms of accuracy and timeliness.

REFERENCES

- [1] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 1750–1762.
- [2] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, and M. R. Lyu, "Heterogeneous anomaly detection for software systems via semi-supervised cross-modal attention," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 1724–1736.
- [3] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2023, pp. 553–565. [Online]. Available: <https://doi.org/10.1145/3611643.3616249>
- [4] Y. Wang, Z. Zhu, Q. Fu, Y. Ma, and P. He, "MRCA: Metric-level root cause analysis for microservices via multi-modal data," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2024, pp. 1057–1068. [Online]. Available: <https://doi.org/10.1145/3691620.3695485>
- [5] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Maat: Performance metric anomaly anticipation for cloud services with conditional diffusion," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2024, pp. 116–128. [Online]. Available: <https://doi.org/10.1109/ASE56229.2023.00082>
- [6] X. Guo et al., "Graph-based trace analysis for microservice architecture understanding and problem diagnosis," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 1387–1397. [Online]. Available: <https://doi.org/10.1145/3368089.3417066>
- [7] Z. Zeng et al., "TraceArk: Towards actionable performance anomaly alerting for online service systems," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 258–269.
- [8] Z. Xie et al., "Unsupervised anomaly detection on microservice traces through graph VAE," in *Proc. ACM Web Conf.*, 2023, pp. 2874–2884.
- [9] P. Liu et al., "Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng.*, 2020, pp. 48–58.
- [10] G. Yu et al., "MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments," in *Proc. Web Conf.*, 2021, pp. 3087–3098. [Online]. Available: <https://doi.org/10.1145/3442381.3449905>
- [11] G. Yu, Z. Huang, and P. Chen, "TraceRank: Abnormal service localization with dis-aggregated end-to-end tracing data in cloud native systems," *J. Softw. Evol. Process.*, vol. 35, no. 10, Oct. 2023, Art. no. e2413 [Online]. Available: <https://doi.org/10.1002/smr.2413>

- [12] L. Huang and T. Zhu, "tprof: Performance profiling via structural aggregation and automated analysis of distributed systems traces," in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 76–91.
- [13] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection from system tracing data using multimodal deep learning," in *Proc. IEEE 12th Int. Conf. Cloud Comput.*, 2019, pp. 179–186.
- [14] J. Chen, F. Liu, J. Jiang, and S. Zhong, "TraceGra: A trace-based anomaly detection for microservice using graph deep learning," *Comput. Commun.*, vol. 204, no. C, pp. 109–117, Apr. 2023. [Online]. Available: <https://doi.org/10.1016/j.comcom.2023.03.028>
- [15] S. Zhang et al., "Efficient and robust trace anomaly detection for large-scale microservice systems," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng.*, 2023, pp. 69–79.
- [16] S. Kong, J. Ai, M. Lu, and Y. Gong, "GRAND: GAN-based software runtime anomaly detection method using trace information," *Neural Netw.*, vol. 169, no. C, pp. 365–377, Jan. 2024. [Online]. Available: <https://doi.org/10.1016/j.neunet.2023.10.036>
- [17] J. Yang, Y. Guo, Y. Chen, Y. Zhao, Z. Lu, and Y. Liang, "Robust anomaly diagnosis in heterogeneous microservices systems under variable invocations," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 2722–2727.
- [18] Y. Luo et al., "Integrating system state into spatio-temporal graph neural network for microservice workload prediction," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 5521–5531. [Online]. Available: <https://doi.org/10.1145/3637528.3671508>
- [19] Z. Wang et al., "Revisiting VAE for unsupervised time series anomaly detection: A frequency perspective," in *Proc. ACM Web Conf.*, 2024, pp. 3096–3105. [Online]. Available: <https://doi.org/10.1145/3589334.3645710>
- [20] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013, *arXiv:1312.6114*. [Online]. Available: <https://api.semanticscholar.org/CorpusID216078090>
- [21] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 574.
- [22] Y. Song and S. Ermon, "Improved techniques for training score-based generative models," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 1043.
- [23] K. Rasul, C. Seward, I. Schuster, and R. Vollgraf, "Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8857–8868.
- [24] X. Zhou et al., "Benchmarking microservice systems for software engineering research," in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 323–324.
- [25] Y. Gan, Y. Zhang, D. Cheng, and e. shetty, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. Int. Conf. Architect. Support Program. Lang. Operating Syst.*, 2019, pp. 3–18.
- [26] Z. Xie et al., "From point-wise to group-wise: A fast and accurate microservice trace anomaly detection approach," in *Proc. 31st ACM Joint Eur. Softw. Eng. Symp. Found. Softw. Eng.*, 2023, pp. 1739–1749.
- [27] Z. Li et al., "Practical root cause localization for microservice systems via trace analysis," in *Proc. IEEE/ACM 29th Int. Symp. Qual. Serv.*, 2021, pp. 1–10.
- [28] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 27268–27286.
- [29] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1003–1012.
- [30] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder–decoder approaches," in *Proc. 8th Workshop Syntax Semantics Struct. Stat. Transl.*, 2014, pp. 103–111. [Online]. Available: <https://api.semanticscholar.org/CorpusID11336213>
- [31] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," 2022, *arXiv:2010.02502*. [Online]. Available: <https://arxiv.org/abs/2010.02502>
- [32] A. I. Jordan, F. Kruger, and S. Lerch, "Evaluating probabilistic forecasts with scoringrules," *J. Stat. Softw.*, vol. 90, pp. 1–37, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID88515280>
- [33] E. de Bézenac, S. S. Rangapuram, and A. Benidis, "Normalizing Kalman filters for multivariate time series analysis," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 252.
- [34] A. Marsick, H. André, I. Khelf, Q. Leclère, and J. Antoni, "Benefits of Mann–Kendall trend analysis for vibration-based condition monitoring," *Mech. Syst. Signal Process.*, vol. 216, 2024, Art. no. 111486.
- [35] Y. Chen et al., "Outage prediction and diagnosis for cloud service systems," in *Proc. World Wide Web Conf.*, 2019, pp. 2659–2665.
- [36] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [37] Jaeger, "Jaeger," 2022. [Online]. Available: <https://www.jaegertracing.io/>
- [38] X. Yuan and Y. Qiao, "Diffusion-TS: Interpretable diffusion for general time series generation," in *Proc. Int. Conf. Learn. Representations*, vol. 2024, 2024, pp. 41582–41610. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/b5b66077d016c037576cc56a82f97f66-Paper-Conference.pdf
- [39] J. Crabbe, N. Huynh, J. P. Stanczuk, and M. van der Schaar, "Time series diffusion in the frequency domain," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, Art. no. 374.
- [40] S. Jeong, J. Sohn, J. Jeon, and H.-I. Suk, "Frequency-conditioned diffusion models for time series generation," in *Proc. 34th ACM Int. Conf. Inf. Knowl. Manage.*, 2025, pp. 1114–1123. [Online]. Available: <https://doi.org/10.1145/3746252.3761050>
- [41] J. Huang, Y. Yang, H. Yu, J. Li, and X. Zheng, "Twin graph-based anomaly detection via attentive multi-modal learning for microservice system," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2023, pp. 66–78.
- [42] B. Yu et al., "Deep learning or classical machine learning? An empirical study on log-based anomaly detection," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–13. [Online]. Available: <https://doi.org/10.1145/3597503.3623308>
- [43] C. Zhang, T. Jia, G. Shen, P. Zhu, and Y. Li, "MetaLog: Generalizable cross-system anomaly detection from logs with meta-learning," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, Art. no. 154. [Online]. Available: <https://doi.org/10.1145/3597503.3639205>
- [44] S. Zhang et al., "End-to-end AutoML for unsupervised log anomaly detection," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2024, pp. 1680–1692. [Online]. Available: <https://doi.org/10.1145/3691620.3695535>
- [45] C. Zhang et al., "TraceCRL: Contrastive representation learning for microservice trace analysis," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 1221–1232. [Online]. Available: <https://doi.org/10.1145/3540250.3549146>
- [46] C. Zhang et al., "DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng.*, 2022, pp. 623–634.



Kaiqi Ding received the MS degree from the National University of Defense Technology, Changsha, China. She is currently working toward the PhD degree with the School of Computer Science, Peking University, Beijing, China. Her research interests include graph neural networks, intelligent operation and maintenance (AIOPs), and fault diagnosis.



Dezhi Ran received the BS (summa cum laude) degree from Peking University, in 2021. He is currently working toward the PhD degree with the School of Computer Science, Peking University, China. His research interests include software engineering, reinforcement learning, and RISC-V system software.



Yuanmu Ma received the MS degree from the School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China, in 2025. Her research interests include graph neural networks and anomaly detection.



Zijian Song received the BS degree from Peking University, Beijing, China, in 2022. He is currently working toward the PhD degree in computer science with Peking University. His research interests include recommender systems, LLM post-training, and LLM reasoning.



Kaigui Bian (Senior Member, IEEE) received the BS degree in computer science from Peking University, Beijing, China, in 2005, and the PhD degree in computer engineering from Virginia Tech, Blacksburg, VA, USA, in 2011. He was a Visiting Young Faculty with Microsoft Research Asia, Beijing, in 2013. His research interests include machine learning, cloud computing, and mobile computing. He received the Best Paper Awards of international conferences. He was a recipient of the IEEE Communication Society Asia-Pacific Board Outstanding Young Researcher

Award in 2018. He serves as an editor of the *IEEE Transactions on Vehicular Technology* and the Organizing Committee member and a Technical Program Committee member for many international conferences.



Tao Xie (Fellow, IEEE) is a Peking University chair professor, Fudan University adjunct top-talent professor, dean of Fudan University Institute of Systems for Advanced Computing (FiSAC), chair of the Department of Software Science and Engineering, School of Computer Science, Peking University, chair and director of Beijing Tongming Lake Information Technology Application Innovation Center (TongmingLAIC), dean of Shanghai Institute of Systems for Open Computing (iSOC), secretary general of Shanghai Open Processor Innovation Center (SOPIC), and chief scientist of Beijing Institute of Open Source Chip. His main research interests include software engineering, system software, software security, trustworthy AI, RISC-V software ecosystem. He is a foreign member of Academia Europaea and a fellow of the ACM, AAAS, Chinese Institute of Electronics (CIE), and China Computer Federation (CCF).